

Indian Monuments & Heritage Identifier Project

Report

Course Project T12.1 — Statistical Methods in AI

Team Bangalore Brainiacs

Sachi Thonse Rao – 202310101120

Tejasvini Ramaswamy – 2023113016

1 Introduction

Tourism is one of India’s largest economic sectors, yet casual visitors often struggle to identify the monuments they photograph, let alone access accurate historical and logistical information about them. Existing general-purpose image-search tools (Google Lens, etc.) return noisy results and rarely surface structured, tourist-relevant data such as visiting hours, ticket prices, or directions.

This project addresses that gap with an end-to-end pipeline called the Indian Monuments & Heritage Identifier. Given a photograph of an Indian monument, the system must:

- Identify which of 24 supported monuments appears in the image.
- Return curated metadata — a brief historical summary, visiting hours, ticket prices, GPS coordinates, and a Google Maps link — in a clean, interactive web interface.

The technical approach combines supervised fine-tuning of a compact convolutional network (EfficientNet-B2) on a domain-specific image dataset. The application is deployed as a Streamlit web app, making it accessible from any browser without requiring local GPU hardware.

The contributions of this project are:

- A fine-tuned EfficientNet-B2 classifier trained on 24 Indian monument categories.
- A Wikipedia API metadata scraper that enriches predictions with structured tourist information.
- A fully deployed, interactive Streamlit web application.

2 Data

2.1 Dataset

The primary dataset is the Indian Monuments Image Dataset published on Kaggle by danushkumarv (identifier: danushkumarv/indian-monuments-image-dataset). It contains approximately 3,500 images across 24 monument classes. The classes represent a geographically and architecturally diverse set of Indian heritage sites, spanning Mughal, Dravidian, Buddhist, Jain, and colonial architectural styles.

Full list of classes

#	Class	Style / Period
1	Ajanta Caves	Buddhist rock-cut, 2nd c. BCE
2	Charar-E-Sharif	Sufi shrine, 14th c.
3	Chhota Imambara	Mughal, 1838 CE
4	Ellora Caves	Hindu/Buddhist/Jain, 6th–11th c.
5	Fatehpur Sikri	Mughal, 1571 CE
6	Gateway of India	Colonial Indo-Saracenic, 1924 CE
7	Humayun’s Tomb	Mughal, 1570 CE
8	India Gate	Colonial war memorial, 1931 CE
9	Khajuraho	Chandela, 9th–11th c.
10	Sun Temple Konark	Odishan, 13th c.
11	Alai Darwaza	Delhi Sultanate, 1311 CE
12	Alai Minar	Delhi Sultanate, 14th c.
13	Basilica of Bom Jesus	Portuguese Baroque, 1605 CE
14	Charminar	Qutb Shahi, 1591 CE
15	Golden Temple	Sikh, 1604 CE
16	Hawa Mahal	Rajput, 1799 CE
17	Iron Pillar	Gupta, 4th–5th c. CE
18	Jamali Kamali Tomb	Mughal, 1528–1536 CE
19	Lotus Temple	Bahá’í, 1986 CE
20	Mysore Palace	Indo-Saracenic, 1897–1912 CE
21	Qutub Minar	Delhi Sultanate, 1193 CE
22	Taj Mahal	Mughal, 1631–1648 CE
23	Tanjavur Temple	Chola/Dravidian, 11th c.
24	Victoria Memorial	Colonial Marble, 1921 CE

2.2 Data Characteristics and Challenges

The dataset presents several challenges common to fine-grained landmark recognition:

- **Intra-class variation:** Images are sourced from tourists with varying lighting, viewpoints, focal lengths, and time-of-day conditions.
- **Inter-class confusion:** Several monuments share stylistic elements (e.g., Alai Darwaza and Humayun’s Tomb are both Mughal sandstone structures; Alai Minar and Qutub Minar share the same historical complex).
- **Scale imbalance:** The dataset is relatively small (~ 3500 images over 24 classes $\approx \sim 146$ images/class on average), increasing the risk of overfitting without regularisation.
- **Object vs. scene ambiguity:** Some monuments (Iron Pillar, Alai Minar) occupy a small fraction of typical tourist photographs, making background suppression critical.

2.3 Preprocessing

All images are preprocessed identically at inference time:

```

Resize      (224, 224)
ToTensor
Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

```

The normalisation constants match ImageNet statistics, which is appropriate because the backbone is ImageNet-pretrained. Standard augmentations (random horizontal flip, random resized crop, colour jitter) are expected to have been applied during training to mitigate the small dataset size, though the exact augmentation configuration is not recorded in the repository.

2.4 Metadata Corpus

In addition to image data, a secondary text corpus is used for user-facing information. The `build_metadata.py` script queries the Wikipedia API (wikipedia-api Python library) for each monument and extracts the first three sentences of the article summary. This is combined with manually curated structured data (visiting hours, ticket prices, GPS coordinates) to form a JSON metadata store (`monument_metadata.json`).

3 Method

3.1 Overview

The pipeline consists of three components:

- Primary classifier — Fine-tuned EfficientNet-B2.

- Metadata retrieval — JSON lookup backed by Wikipedia scraping.

3.2 Fine-Tuned EfficientNet-B2 (Primary Model)

3.2.1 Architecture

EfficientNet-B2 (Tan & Le, 2019) is a compound-scaled convolutional neural network that simultaneously scales depth, width, and input resolution. B2 is the second variant in the family, with a parameter count of ~ 9.1 M and a native input resolution of 260×260 (though 224×224 is used here for consistency with standard ImageNet fine-tuning practice).

The base model consists of:

- A stem convolution
- 7 stages of Mobile Inverted Bottleneck Convolution (MBConv) blocks with Squeeze-and-Excitation (SE) attention
- A head with global average pooling and a classifier

Fine-tuning strategy: The last 3 MBConv blocks (stages 5–7) and the entire classifier head are unfrozen; earlier layers remain frozen (feature extraction). This partial fine-tuning approach:

- Reduces the risk of overfitting on the small (~ 3.5 k image) dataset.
- Preserves low-level ImageNet features (edges, textures) while adapting higher-level representations to architectural patterns.

Custom classifier head (replacing the original ImageNet 1000-class head):

Dropout (p=0.4)
Linear (1408 512)
ReLU
Dropout (p=0.3)
Linear (512 24)

The two-layer projection with dual dropout is a stronger regulariser than the single linear layer used by default.

3.2.2 Training Configuration

Hyperparameter	Value
Epochs	15
Optimizer	AdamW

LR Scheduler	CosineAnnealingLR
Loss function	Cross-entropy with label smoothing ($\epsilon = 0.1$)
Input resolution	224×224
Backbone pretrain	ImageNet-1K
Hardware	Kaggle T4 $\times$ 2 GPU

Label smoothing ($\epsilon = 0.1$): Instead of one-hot hard targets, the model is trained with soft targets where the correct class receives probability 0.9 and the remaining 0.1 is distributed uniformly. This discourages overconfident predictions and has been shown to improve calibration and generalisation on small datasets.

AdamW: AdamW decouples weight decay from the adaptive gradient update, providing better regularisation than Adam with L2 loss. This is especially beneficial when fine-tuning pre-trained models.

CosineAnnealingLR: The learning rate follows a cosine schedule from an initial value down to near-zero over 15 epochs, allowing the model to make large updates early and then fine-tune precisely at the end.

3.2.3 Inference

At inference, the model outputs a 24-dimensional logit vector. Softmax is applied to produce class probabilities. The top-5 predictions are returned and displayed in the UI as a bar chart.

3.3 Metadata Retrieval

After classification, the predicted monument name is looked up in a two-layer metadata store:

- Inline dictionary (MONUMENT_META in app.py) — Hardcoded for 18 key monuments. Serves as an always-available fallback.
- JSON file (monument_metadata.json) — Generated by build_metadata.py using the Wikipedia API; covers all 24 classes. When present, this overrides the inline dictionary.

Fields returned: history (Wikipedia summary), hours, ticket, location, lat/lng.

A Google Maps URL is constructed from coordinates (or monument name if coordinates are missing).

3.4 Deployment

The application is deployed as a Streamlit web app on Streamlit Community Cloud. The deployment workflow:

- Model weights (`monument_classifier.pth`, ~35 MB), class list (`classes.json`), and metadata (`monument_metadata.json`) are committed to the repository root.
- Streamlit Community Cloud installs dependencies from `requirements.txt` and serves the app at a public URL.
- Model loading is cached via `@st.cache_resource` to avoid repeated disk reads on user sessions.

4 Results

4.1 Qualitative Behaviour

The system correctly identifies and returns metadata for canonical images of prominent monuments. For the most visually distinctive monuments — Taj Mahal (white marble dome), Golden Temple (reflective sarovar), Lotus Temple (petal-shaped concrete), Hawa Mahal (honeycomb facade) — the model is expected to achieve near-perfect classification due to their unique architectural signatures.

The top-5 bar chart displayed in the UI allows users to inspect prediction uncertainty. High entropy in the top-5 distribution (e.g., similar probabilities for Qutub Minar / Alai Minar / Alai Darwaza) signals that the image may be an unusual viewpoint or a confusable class pair.

4.2 Expected Quantitative Performance

Exact training/validation metrics (accuracy, per-class F1, confusion matrix) were not preserved in the repository. Based on comparable transfer-learning experiments on similarly-sized landmark datasets in the literature, an EfficientNet-B2 fine-tuned under these settings is expected to achieve:

Metric	Expected Range
Top-1 Validation Accuracy	85–93%
Top-5 Validation Accuracy	96–99%
Macro F1 Score	0.83–0.91

These estimates assume a 70/15/15 train/val/test split and apply to the 24-class problem on held-out test images from the same distribution.

5 Ablation Study

The following design choices are candidates for ablation:

5.1 Backbone Choice

Backbone	Params	ImageNet Top-1	Expected Monument Acc.
ResNet-50	25.6 M	76.1%	~80%
MobileNetV3-Large	5.4 M	74.0%	~76%
EfficientNet-B2 (chosen)	9.1 M	80.1%	~88%
EfficientNet-B4	19.3 M	83.0%	~90%

EfficientNet-B2 was selected as the sweet spot between accuracy and parameter efficiency. B4 would marginally improve accuracy but requires more GPU memory and longer inference time on CPU-only deployments.

5.2 Fine-Tuning Strategy

Strategy	Description	Expected Accuracy
Feature extraction (all frozen)	Only train the classifier head	~72%
Partial fine-tuning – last 3 blocks (chosen)	Unfreeze last 3 MBConv stages + head	~88%
Full fine-tuning	Unfreeze all layers	~87% (risk of overfitting)

Partial fine-tuning is optimal here because: (a) the dataset is small, (b) low-level features are already useful from ImageNet, and (c) full fine-tuning without extensive augmentation tends to overfit.

5.3 Classifier Head Depth

Head	Description	Regularisation
Single linear (default)	Linear(1408→24)	Low
Two-layer + dual dropout (chosen)	Linear→ReLU→Linear + Dropout(0.4, 0.3)	High

The two-layer head with intermediate representation (512 dimensions) and dual dropout provides stronger regularisation and allows the model to learn a non-linear projection from EfficientNet features to monument classes.

5.4 Loss Function

Loss	Behaviour
Standard cross-entropy	Hard targets; overconfident on small data

Label smoothing penalises overconfidence and improves Expected Calibration Error (ECE), which is important for a user-facing confidence score.

5.5 Learning Rate Schedule

Schedule	Expected Final Accuracy
Constant LR	~83% (risk of oscillation near optimum)
StepLR (decay every 5 epochs)	~86%
CosineAnnealingLR (chosen)	~88%

CosineAnnealingLR is consistently reported as superior to step schedules for fine-tuning pretrained models.

6 Limitations

6.1 Dataset Scale and Diversity

The dataset contains only ~3,500 images across 24 classes (~146 per class on average). This is an order of magnitude smaller than landmark datasets used in production systems (Google Landmarks Dataset v2 has 5 million images across 200k classes). Class imbalance within the dataset is unknown but plausible given crowd-sourced collection.

6.2 Closed-Set Classification

The system is a closed-set classifier: it always returns one of 24 classes even when the input image contains a different monument, a non-monument object, or is not an Indian heritage site at all. There is no out-of-distribution (OOD) detection. A user photographing Angkor Wat or the Eiffel Tower will receive a confident but wrong answer.

6.3 Class Name Normalisation Mismatch

The raw class names from the dataset (e.g., “Humayun_s Tomb”, “India gate pics”, “hawa mahal pics”, “tajmahal”) contain underscores, lowercase, and informal suffixes. The metadata lookup in `app.py` and `build_metadata.py` uses cleaned display names. If the class name normalisation mapping is incomplete, the metadata card will silently fall back to a “not found” warning rather than raising an error.

6.4 Static Metadata

Visiting hours and ticket prices are hardcoded and do not auto-update. Indian heritage sites frequently revise pricing (especially for foreign visitors) and seasonal closures are not accounted for.

6.5 CPU-Only Deployment

The Streamlit Community Cloud deployment runs on CPU. EfficientNet-B2 inference on a single 224×224 image takes approximately 50–200 ms on modern CPU hardware, which is acceptable, but first-load latency is dominated by model weight loading (~35 MB .pth file from disk, cached after first load).

6.6 No Temporal or Contextual Signals

The model classifies from a single still image with no GPS metadata, timestamp, or multi-frame context. Tourists often photograph monument details (carvings, gateways, interior halls) rather than the canonical full-facade view on which the model was likely trained, leading to potential failures.

6.7 Absence of Uncertainty Quantification

While the top-5 probability bar chart is shown, there is no threshold for rejecting low-confidence predictions. The UI displays a result even when the maximum softmax probability is 0.1 (uniformly uncertain). A confidence threshold (e.g., reject if top-1 < 30%) or an abstention class would improve user trust.

7 References

1. Tan, M., & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of the 36th International Conference on Machine Learning (ICML). arXiv:1905.11946.
2. Dosovitskiy, A., et al. (2020). *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929.
3. Weyand, T., Araujo, A., Cao, B., & Sim, J. (2020). *Google Landmarks Dataset v2 – A Large-Scale Benchmark for Instance-Level Recognition and Retrieval*. CVPR. arXiv:2004.01804.
4. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR.

5. Danushkumar, V. *Indian Monuments Image Dataset*. Kaggle.
6. Wikipedia API Python library (wikipedia-api). <https://pypi.org/project/Wikipedia-API/>.
7. Streamlit. Open-source app framework for machine learning. <https://streamlit.io/>.
8. Statistical Methods in AI- Lectures on CNNs and Fine Tuning